

Coasterbot

Softwaredokumentation



Dennis Borutta, B. Eng	dennis.borutta@outlook.com
Stefan Etringer, B. Eng	mail@stefan.works
Gereon Such, M. Sc.	gereonsuch@gmail.com

Versionshistorie

Version	Datum	Bearbeiter	Änderungen
1	08.09.2026	Etringer	Erstfassung der Softwaredokumentation der Simulationssoftware
2	08.09.2026	Etringer	Abschnitt 5 (Lokalisierung) um Prinzip (Kopplnavigation) und Grenzen der Lokalisierung ergänzt
3	13.09.2026	Etringer	Offene Punkte aus der Hardware-Rückmeldung zu RobotHAL ergänzt (Abschnitt 4.3)
4	13.09.2026	Etringer	Entscheidung umgesetzt: Kantensensoren liefern jetzt <code>bool</code> statt <code>float</code> ; Kalibrierung in WebotsHAL gekapselt
5	13.09.2026	Etringer	Entscheidung umgesetzt: <code>enum WheelId</code> auf <code>WHEEL_LEFT/WHEEL_RIGHT</code> reduziert (reale Elektronik hat nur einen Encoder je Seite)
6	14.09.2026	Etringer	Geglättete Kartenanpassung (<code>OccupancyGrid::reportOccupied</code>), digitales Kantensignal (<code>EDGE_ACTIVE_HIGH</code>) umgesetzt; Ausblick „initiale Tischerkundung“ ergänzt
7	14.09.2026	Etringer	Ausblick „initiale Tischerkundung“ zu einem konkreten Stufenplan (Stufe 0 bis 7) ausgearbeitet
8	16.09.2026	Etringer	Abschnitt 4.3 (Gyro-Schnittstelle, Erreichbarkeit einer definierten Winkelgeschwindigkeit) um konkrete Hardware-Hinweise ergänzt (Kalibrierung, Arduino-Bibliotheken, Skid-Steer-Fahrverhalten)

Inhaltsverzeichnis

1	Einleitung	5
1.1	Zweck des Dokuments	5
1.2	Gegenstand und Abgrenzung	5
1.3	Begriffe und Abkürzungen	6
2	Softwarearchitektur	6
2.1	Architekturprinzip	6
2.2	Abbildung auf die Pflichtenheft-Architektur	7
2.3	Datenfluss im kombinierten Betrieb	8
3	Laufzeitumgebung und Projektstruktur	9
3.1	Werkzeuge	9
3.2	Quelldateien	9
4	Hardwareabstraktion	10
4.1	Interface RobotHAL	10
4.2	Implementierung WebotsHAL	11
4.3	Offene Punkte aus der Hardware-Rückmeldung	12
5	Lokalisierungskomponente	14
5.1	Prinzip	14
5.2	Verfahren	15
5.3	Schnittstelle	16
5.4	Grenzen der Lokalisierung	16
6	Umgebungsmodell	16
6.1	Aufbau	16
6.2	Funktionen	17
6.3	Geglättete Live-Hindernismeldung	17
7	Globale Pfadplanung mit A*	18
7.1	Algorithmus	18
7.2	Auswahlbegründung	19
8	Dynamische Neuplanung mit D* Lite	19
8.1	Verfahren	19
9	Lokale Kollisionsvermeidung und Bewegungssteuerung mit DWA	20
9.1	Dynamic Window Approach	20
9.2	Pfadverfolgung (Pure Pursuit)	20
9.3	Hindernisse	20
9.4	Bewegungssteuerung	21

10 Sicherheitskomponente	21
10.1 Funktion	21
10.2 Schnittstelle	22
11 Reaktive Verhaltenslogik	22
12 Ablaufsteuerung und Betriebsarten	23
12.1 Ablauf MODE_SIM_TEST	23
13 Simulationsmodell	24
13.1 Robotermodell (PROTO)	24
13.2 Welten	24
13.3 Reproduzierbarkeit	25
14 Systemüberwachung	25
15 Abbildung auf Anforderungen und Testfälle	26
15.1 Status der Simulationstestfälle	27
16 Grenzen und offene Punkte	28
16.1 Ausblick: initiale Tischerkundung	28
17 Anhang A: Schnittstelle RobotHAL	30
18 Anhang B: Wesentliche Parameter	31

1 Einleitung

1.1 Zweck des Dokuments

Dieses Dokument beschreibt die **Software der Simulation des Coasterbots** sowie deren Funktionen. Dokumentiert wird die Softwarearchitektur, die einzelnen Softwarekomponenten, ihre Schnittstellen und ihr Laufzeitverhalten in der Simulationsumgebung.

Grundlage der Dokumentation sind die im Projekt erstellten Anforderungs- und Testdokumente:

- **Lastenheft:** funktionale und nichtfunktionale Anforderungen des Auftraggebers (Kennungen NAV-*, SAF-*, MON-*, NFA-*, SIM-*).
- **Pflichtenheft:** die geplante technische Umsetzung, insbesondere die geschichtete System- und Softwarearchitektur (Hardwaretreiber, Hardwareabstraktion, Systemdienste, Funktionskomponenten, Anwendungsebene, Simulationsebene).
- **Testkonzept:** Teststrategie und Simulationstestfälle (ST-SIM-001 bis ST-SIM-008).
- **Alternativenbeschreibung:** Auswahl der Navigations- und Lokalisierungsalgorithmen (A*, D* Lite, DWA, Sensorfusion).

Die Software wurde entsprechend der im Pflichtenheft beschriebenen Abstraktion umgesetzt: Die eigentliche Systemlogik ist vollständig von der konkreten Hardware entkoppelt und kommuniziert über eine Hardwareabstraktionsschicht. Dadurch sind die Funktionen im Quellcode sowohl in der Simulation (Webots) als auch auf der Zielhardware (Mikrocontroller) verwendbar. lauffähig.

1.2 Gegenstand und Abgrenzung

Gegenstand dieser Dokumentation ist die Software des Webots-Controllers **coasterbot** (C++), die den simulierten Roboter steuert. Dokumentiert werden:

- die Hardwareabstraktion (**RobotHAL** / **WebotsHAL**),
- die Lokalisierung (**PoseEstimator**),
- das Umgebungsmodell (**OccupancyGrid**),
- die globale Pfadplanung (**astarPlan**, A*),
- die dynamische Neuplanung (**DStarLite**, D* Lite),
- die lokale Kollisionsvermeidung und Bewegungssteuerung (**DwaPlanner**),
- die Sicherheitskomponente (**SafetyMonitor**),
- die reaktive Verhaltenslogik (**RobotLogic**),
- die Ablaufsteuerung und Betriebsarten (**main.cpp**),
- das Simulationsmodell (Webots-Welten und **Coasterbot-PROTO**).

Nicht Bestandteil dieser Dokumentation bzw. der aktuellen Umsetzung sind:

- das Untersetzerhandling (Aufnahme/Ablage von Getränkeuntersetzern, CST-*) einschließlich der Manipulationsplanung (IK + RRT*),
- die missionsbezogene Ablaufsteuerung `IDLE` → `SEARCH_COASTER` → `PICK_UP` → `TRANSPORT` → `PLACE` → `VERIFY` → `DONE`,
- eine probabilistische Sensorfusion (EKF/UKF), statt dessen wird ein Komplementärfilter verwendet (siehe Abschnitt 5),
- die Fehlerinjektionstests ST-SIM-005 (Sensorausfall) und ST-SIM-006 (Aktorfehler).

Der Umsetzungsstand bezieht sich auf **Maturity Level 1** (Prototyp): autonome Navigation, Lokalisierung, sichere Bewegung und Tischkantenerkennung.

1.3 Begriffe und Abkürzungen

Begriff	Bedeutung
HAL	Hardware Abstraction Layer, Hardwareabstraktionsschicht
PROTO	Webots-Modellbeschreibung eines Roboters oder Objekts
Pose	Lage des Roboters in der Ebene: (x, y, θ)
Belegungskarte	Rasterkarte der Fläche, Zellen frei oder belegt (occupancy grid)
A*	heuristischer Graphsuch-Algorithmus für die globale Pfadplanung
D* Lite	inkrementeller Neuplaner (Koenig & Likhachev, 2002)
DWA	Dynamic Window Approach, lokale Bewegungsplanung im Geschwindigkeitsraum
Carrot-Punkt	vorausliegender Zielpunkt auf dem Pfad (Pure Pursuit)
Skid-Steer	Panzerlenkung: Lenkung über Drehzahldifferenz linker/rechter Räder
Regelzyklus	ein Simulationsschritt (<code>basicTimeStep</code>), typ. 32 ms

2 Softwarearchitektur

2.1 Architekturprinzip

Die Software folgt dem im Pflichtenheft (Kapitel „Softwarekonzept“) beschriebenen **geschichteten, komponentenbasierten** Architekturmodell. Jede Schicht hat eine klar abgegrenzte Verantwortlichkeit und kommuniziert über festgelegte Schnittstellen mit den benachbarten Schichten. Höhere Schichten kennen keine Hardware- oder Simulationsdetails.

Die zentrale Abstraktion ist die Hardwareabstraktionsschicht **RobotHAL**. Sie ist ein rein abstraktes C++-Interface, das *weder* Webots *noch* eine konkrete Mikrocontroller-Bibliothek kennt. Alle Funktionskomponenten (Lokalisierung, Planung, Sicherheit) sind

gegen dieses Interface programmiert. Dadurch werden die Anforderungen NFA-ARC-005, NFA-MAI-001/002 und NFA-SIM-004 erfüllt.

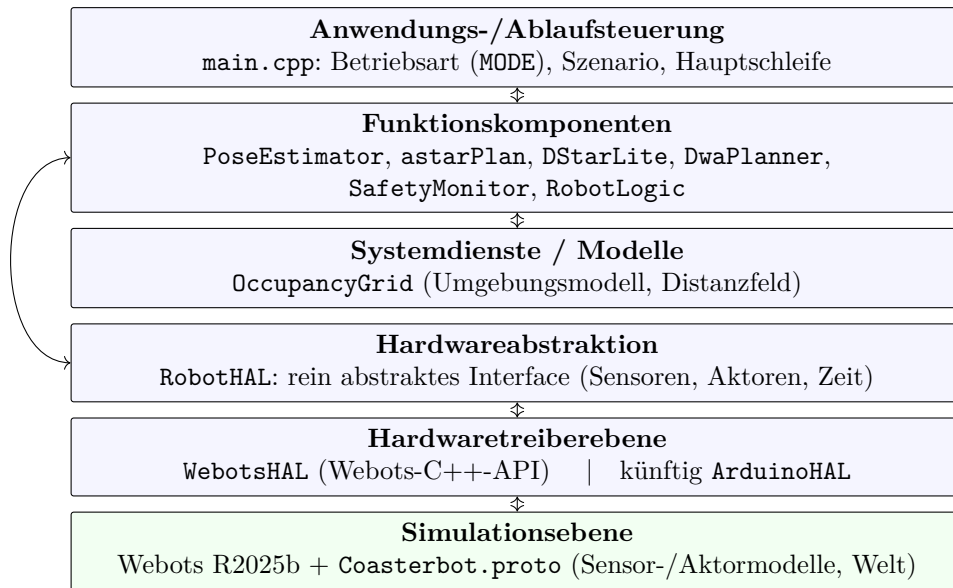


Abbildung 1: Schichtenmodell der Simulationssoftware. Die Funktionskomponenten greifen über RobotHAL auf Sensoren und Aktoren zu.

2.2 Abbildung auf die Pflichtenheft-Architektur

Tabelle 2 ordnet die im Pflichtenheft definierten Architekturebenen und Funktionskomponenten den konkreten Quelldateien des Simulations-Controllers zu.

Tabelle 2: Zuordnung Pflichtenheft-Architektur → Umsetzung.

Pflichtenheft	Umsetzung (Datei / Klasse)	Anforderung
Hardwaretreiberebene	webots_hal.cpp (Geräte-Handles)	keine
Hardwareabstraktions- ebene	robot_hal.h (RobotHAL), webots_hal (WebotsHAL)	NFA-ARC-005
Systemdienste / Umgebungsmodell	occupancy_grid (OccupancyGrid)	SIM-004
Navigationskomponente	astar (A*), dstar_lite (D* Lite)	NAV-005/008/009
Lokalisierungs- komponente	pose_estimator (PoseEstimator)	NAV-006, NAV-007
Bewegungssteuerung	dwa_planner (DwaPlanner::driveVW), robot_hal.h	HW-ACT-002/003, SAF-004

Pflichtenheft	Umsetzung (Datei / Klasse)	Anforderung
Sicherheitskomponente / Sicherheitslogik Systemüberwachung	<code>safety_monitor</code> (<code>SafetyMonitor</code>) Konsolenprotokoll je Betriebsart in <code>main.cpp</code>	NAV-003/004, SAF-001..005 MON-001..005
Anwendungsebene	<code>main.cpp</code> (MODE, Szenarien, Hauptschleifen)	keine
Simulationsebene	<code>worlds/*.wbt</code> , <code>Coasterbot.proto</code>	NFA-SIM-001..005
Reaktive Verhaltenslogik (Bring-up)	<code>robot_logic</code> (<code>RobotLogic</code>)	NAV-001/002, SAF-003

2.3 Datenfluss im kombinierten Betrieb

Abbildung 2 zeigt den Datenfluss der Betriebsart `MODE_SIM_TEST`, in der alle wesentlichen Komponenten zusammenwirken (Grundlage der Testfälle `ST-SIM-001` bis `ST-SIM-004`).

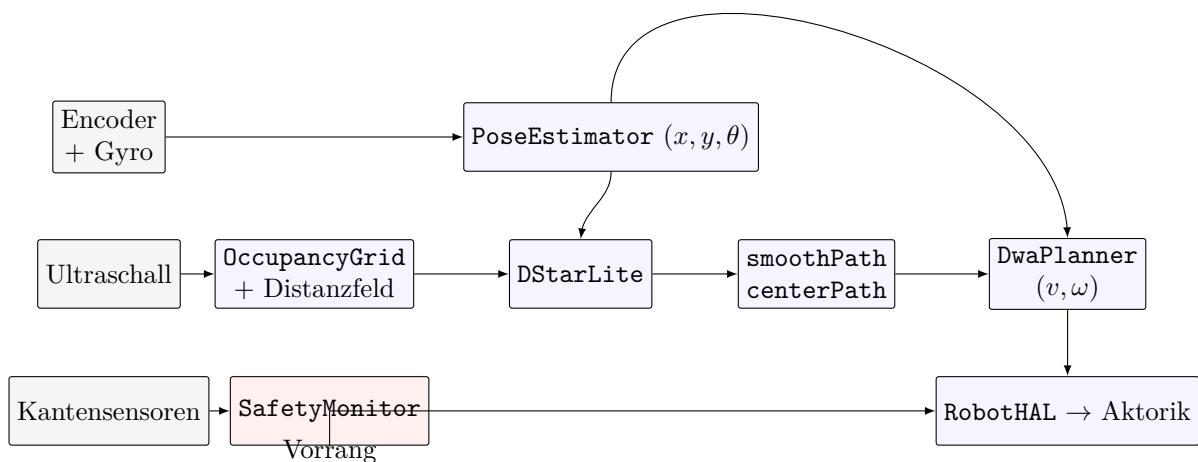


Abbildung 2: Datenfluss in `MODE_SIM_TEST`. Die Sicherheitskomponente wird in jedem Regelzyklus *vor* der Navigation ausgewertet und übernimmt bei einem Kantenereignis sofort die Aktorik.

Ablauf eines Regelzyklus:

1. `hal.step()`: Simulationsschritt, neue Sensorwerte.
2. `pose.update()`: Pose aus Radwinkeln und Gyro fortschreiben.

3. `safety.update()`: Kantensensoren prüfen; hält die Sicherheitskomponente die Aktorik, endet der Zyklus hier.
4. Kartenpflege: Ein per Ultraschall erkanntes, unkartiertes Hindernis bzw. eine erkannte Kante wird in die Belegungskarte eingetragen und `DStarLite` benachrichtigt.
5. `planner.setStart()` / `computeShortestPath()`: Pfad inkrementell aktualisieren.
6. `smoothPath()` + `centerPath()`: Pfad glätten und in Korridormitte schieben.
7. `dwa.update()`: kollisionsfreies Geschwindigkeitspaar wählen und als Radgeschwindigkeiten ausgeben.

3 Laufzeitumgebung und Projektstruktur

3.1 Werkzeuge

Element	Beschreibung
Simulator	Webots R2025b (Nightly-Build, Commit 033dbaea...)
Sprache	C++ (objektorientiert, eine Klasse je Komponente)
Build	Webots-Makefile.include, je Controller ein Makefile
Robotermodell	<code>protos/coasterbot/Coasterbot.proto</code> (aus CAD abgeleitet)
Controller	<code>controllers/coasterbot/</code> (dieser Quellcode)

Übersetzen und Ausführen:

```
export WEBOTS_HOME=~/.Documents/webots/webots

# nur den Controller bauen
cd controllers/coasterbot && make

# eine Welt starten (GUI)
$WEBOTS_HOME/webots worlds/coasterbot-simtest.wbt

# kopflos / reproduzierbar
$WEBOTS_HOME/webots --mode=fast --no-rendering --stdout --stderr \
    worlds/coasterbot-simtest.wbt
```

3.2 Quelldateien

Tabelle 4: Dateien des Controllers `coasterbot`.

Datei	Inhalt
<code>robot_hal.h</code>	abstraktes Interface <code>RobotHAL</code> , enum <code>WheelId</code>
<code>webots_hal.{h,cpp}</code>	<code>WebotsHAL</code> : <code>RobotHAL</code> , einzige Webots-abhängige Datei
<code>pose_estimator.{h,cpp}</code>	<code>PoseEstimator</code> , Odometrie- und Gyro-Fusion
<code>occupancy_grid.{h,cpp}</code>	<code>OccupancyGrid</code> , Rasterkarte mit Distanzfeld
<code>astar.{h,cpp}</code>	<code>astarPlan</code> , <code>smoothPath</code> , <code>centerPath</code>
<code>dstar_lite.{h,cpp}</code>	<code>DStarLite</code> , inkrementeller Neuplaner
<code>dwa_planner.{h,cpp}</code>	<code>DwaPlanner</code> , lokale Planung und Bewegungssteuerung
<code>safety_monitor.{h,cpp}</code>	<code>SafetyMonitor</code> , Sicherheitskomponente
<code>robot_logic.{h,cpp}</code>	<code>RobotLogic</code> , reaktiver Zustandsautomat
<code>main.cpp</code>	Betriebsarten, Szenarien, Hauptschleifen
<code>Makefile</code>	Liste der zu übersetzenden Quelldateien

4 Hardwareabstraktion

4.1 Interface `RobotHAL`

`RobotHAL` (Datei `robot_hal.h`) ist eine rein abstrakte Klasse. Sie beschreibt ausschließlich, *welche* Fähigkeiten der Roboter besitzt, nicht wie sie technisch realisiert werden. Sie erfüllt NFA-ARC-004 (eindeutig definierte Schnittstellen) und NFA-ARC-005 (Hardwareabstraktion).

Tabelle 5: Methoden von `RobotHAL`.

Methode	Bedeutung / Einheit
<code>bool step()</code>	ein Regelzyklus; <code>false</code> = Simulation beendet
<code>void setLeftSpeed(float)</code>	Sollgeschwindigkeit linke Seite [rad/s], positiv = vorwärts
<code>void setRightSpeed(float)</code>	Sollgeschwindigkeit rechte Seite [rad/s]
<code>float getUltrasonicDistance()</code>	Abstand nach vorn [m], kleiner = näher
<code>bool getEdgeFrontLeft() ...</code>	vier Kantensensoren; <code>true</code> = „kein Boden“ (Kante), bereits kalibriert
<code>float getYaw()</code>	Gierwinkel um die Hochachse [rad]
<code>float getGyroZ()</code>	Giergeschwindigkeit um die Hochachse [rad/s]
<code>float getWheelAngle(WheelId)</code>	akkumulierter Radwinkel EINER SEITE [rad], positiv = vorwärts
<code>float getTime()</code>	Laufzeit seit Programmstart [s]

Konventionen.

- **Vorzeichen:** `setLeftSpeed/setRightSpeed` positiv $\Rightarrow$ Vorwärtsfahrt. Lenkung über die Drehzahldifferenz (Skid-Steer).
- **Radpaarung:** Die beiden Räder einer Seite sind mechanisch gekoppelt und laufen stets gemeinsam (wie beim realen Coasterbot: zwei Getriebemotoren je Seite, gemeinsam angesteuert). Es gibt daher *keinen* Einzelrad-Befehl; `enum WheelId` hat entsprechend nur die zwei Werte `WHEEL_LEFT/WHEEL_RIGHT`. Die reale Elektronik besitzt ebenfalls nur einen Encoder je Seite, nicht einen je Rad (siehe Abschnitt 4.3).
- **Kantensensoren:** `RobotHAL` gibt bereits ein kalibriertes `bool` zurück (`true` = Kante). Die Umrechnung des Rohwerts (in der Simulation ≈ 0 auf einer Fläche, ≈ 1000 über einer Kante; Schwelle 500) ist Aufgabe der jeweiligen HAL-Implementierung und bleibt dort gekapselt (siehe Abschnitt 4.3).

4.2 Implementierung WebotsHAL

`WebotsHAL` (Datei `webots_hal.cpp`) ist die einzige Datei, die die Webots-C++-API einbindet. Bei einer Portierung auf die Zielhardware wird nur diese Datei durch eine `ArduinoHAL` mit identischer Signatur ersetzt; die übrigen Komponenten bleiben unverändert.

Aufgaben:

- Beschaffung der Geräte-Handles über die im PROTO vergebenen Namen: die gekoppelten Motoren `wheel_left::front` und `wheel_right::front`, die vier PositionSensor `wheel*_sensor` (je zwei pro Seite, siehe unten), der Ultraschall `ultrasonic`, die vier Kantensensoren `edge_*`, sowie `inertial_unit` und `gyro`. Die Namensstrings sind der Vertrag zwischen PROTO und Software.
- Betrieb der Antriebsmotoren im Geschwindigkeitsmodus (`setPosition( $\infty$ )`).
- **Vorzeichenkorrektur:** Das Radmodell des PROTO rollt den Roboter bei positiver Motordrehzahl nach $+Z$ (Rückseite). Da die HAL-Konvention „positiv = vorwärts“ lautet und die Vorderseite bei $-Z$ liegt, negiert `WebotsHAL` die Motor- und Encoder-Werte.
- **Gierrate:** Das Y-up-PROTO wird in der Z-up-Welt mit `rotation 1 0 0 1.5708` aufgestellt. Die Welt-Hochachse fällt daher mit der lokalen $+Y$ -Achse des Gyros zusammen; `getGyroZ()` liefert die Y -Komponente des Gyrovektors.
- **Encoder-Aggregation:** Die reale Elektronik besitzt nur einen Encoder je Seite (`RobotHAL::WheelId` kennt entsprechend nur `WHEEL_LEFT/WHEEL_RIGHT`). Das PROTO stellt in der Simulation trotzdem vier unabhängige `PositionSensor` bereit (je Rad); `getWheelAngle()` mittelt Front- und Heckrad je Seite, um nach außen dieselbe Information wie der eine reale Encoder zu liefern.
- **Digitales Kantensignal:** Die `lookupTable` der vier Kantensensoren im PROTO liefert bereits ein digitales Signal (0 oder 1, kein analoger Rohwert; siehe Abschnitt 13.1). `digitalEdge()` liest diesen Pegel und bildet ihn über die `private`

Konstante `EDGE_ACTIVE_HIGH` auf das semantische `bool` ab. Das ist die einzige Stelle im Code, an der bekannt sein muss, ob „Kante“ am Sensor elektrisch HIGH oder LOW bedeutet; eine `ArduinoHAL`, die tatsächlich `digitalRead()` an einem Pin aufruft, bildet dort genauso ab und passt bei einem aktiv-low-Sensormodul nur diese eine Konstante an.

Die coupled motors von Webots setzen einen Befehl an `wheel_left::front` automatisch auch auf `wheel_left::rear` um (Faktor `multiplier = 1`), sodass die Software je Seite nur ein Handle benötigt.

4.3 Offene Punkte aus der Hardware-Rückmeldung

Die folgenden Punkte wurden vom Hardware-Team am 13.09.2026 zur Schnittstelle `RobotHAL` zurückgemeldet. Zwei Punkte (Radgranularität, Kantensensoren) wurden inzwischen entschieden und umgesetzt; die übrigen sind weiterhin **offene Fragen** und hier dokumentiert, damit sie bei der Portierung auf die Zielhardware (`ArduinoHAL`) nicht verloren gehen.

Radgranularität: `enum WheelId` (entschieden). Auch diese Frage ist geklärt: Die reale Elektronik besitzt je Fahrzeugseite nur *einen* Encoder (und ohnehin nur eine Ansteuerung; vorderes und hinteres Rad einer Seite drehen daher zwangsläufig immer gleich schnell, was `setLeftSpeed/setRightSpeed` bereits korrekt abbildet, siehe „Radpaarung“ oben). `enum WheelId` wurde deshalb von vier Werten (`WHEEL_FL/FR/RL/RR`) auf zwei reduziert (`WHEEL_LEFT/WHEEL_RIGHT`), und `PoseEstimator` (Abschnitt 5) liest seither direkt den Seitenwert statt einen Mittelwert aus zwei (zuvor fiktiven) Einzelmessungen zu bilden. Da das PROTO in der Simulation weiterhin vier unabhängige `PositionSensor` bereitstellt (ein Rasteraufbau des CAD-Modells, kein Beleg für vier reale Messpunkte), mittelt `WebotsHAL` Front- und Heckrad je Seite intern, um nach außen exakt die Information eines einzelnen realen Encoders je Seite zu liefern (Abschnitt 4.1). Für eine künftige `ArduinoHAL` entfällt dieser Mittelungsschritt, da sie ohnehin nur einen Encoder je Seite liest.

Kantensensoren: `float` oder `bool`? (entschieden). Diese Frage wurde inzwischen entschieden: `RobotHAL` liefert die vier Kantensensoren seit Version 3 dieses Dokuments als `bool` (`true` = Kante). Zuvor lieferte die Schnittstelle einen rohen `float`, den `RobotLogic` und `SafetyMonitor` unabhängig voneinander gegen dieselbe Konstante `EDGE_THRESHOLD = 500` auswerteten. Für die Algorithmik ist aber tatsächlich nur die boolesche Aussage „Kante / kein Boden“ relevant, und die Kalibrierung eines IR-Kantensensors (Reflexionsgrad des Untergrunds, Umgebungslicht, Sensorabstand) ist Hardwarewissen, das laut Pflichtenheft in die Hardwareabstraktion gehört („Umwandlung hardwareabhängiger Daten in ein einheitliches Format“). Die Kalibrierung sitzt jetzt ausschließlich in `WebotsHAL` (`EDGE_THRESHOLD` als private Konstante, Rohwert > Schwelle $\Rightarrow$ `true`); eine künftige

ArduinoHAL kalibriert ihren eigenen Rohwert ebenso dort. Damit entfällt zugleich die doppelte Ablage derselben Schwelle in `robot_logic.cpp` und `safety_monitor.cpp`: Beide fragen jetzt nur noch `hal.getEdgeFrontLeft() || ...` ab.

Nachtrag: Die Kalibrierung wurde seither noch einen Schritt weitergeführt: die vier Kantensensoren liefern in der Simulation nun ein echtes *digitales* Signal statt eines analogen Rohwerts mit Schwellenwertvergleich, und die Sensor-Polarität (HIGH/LOW) ist über eine einzige Konstante abstrahiert. Details in Abschnitt 4.2 (`digitalEdge()`).

Gyro-Schnittstelle. Ob `getGyroZ()` (ein einzelner Skalar, Giergeschwindigkeit um die Hochachse in rad/s) unverändert zu einer realen IMU passt, ist mangels Hardwareerfahrung noch nicht abschließend bewertet, folgende Leitplanken stehen aber inzwischen fest (Rückmeldung 16.09.2026).

Ein Gyro liefert eine *Winkelgeschwindigkeit*, keinen Winkel; die Pose ändert sich, indem diese Rate über die Zeit integriert wird, genau wie es `PoseEstimator::update()` bereits tut (`dThetaGyro = hal_.getGyroZ() * dt`, Abschnitt 5). Auf realer Hardware wäre dieser Schritt unverändert, die Integration ist kontinuierlich: Anders als ein Encoder, der erst nach einer Mindestdrehung einen Tick meldet, geht jede noch so kleine reale Drehung sofort in den nächsten Integrationsschritt ein. Begrenzend sind nur das Rauschen bzw. die Bias-Stabilität des Sensors und die Abtastrate der Regelschleife.

Neu gegenüber der Simulation ist die **Kalibrierung**: Die Simulation modelliert weder Bias noch Rauschen des Gyros, ein echter MEMS-Gyro hat dagegen im Stillstand einen Rest-Offset ungleich null. Üblich ist eine kurze Startkalibrierung (Roboter für ein bis zwei Sekunden stillhalten, Rohwert mitteln, diesen Bias von jeder folgenden Messung abziehen). Für den verbreiteten MPU-6050 (I²C, 6-Achsen Gyro und Beschleunigungssensor) existieren dafür fertige Arduino-Bibliotheken: `Adafruit_MPU6050` mit `Adafruit_Sensor` (einfache API, kalibrierte rad/s) oder die tiefer gehende `I2Cdevlib`-Bibliothek (nutzt den chipinternen DMP für Sensorfusion bereits auf dem IMU). Eine teurere Alternative wie der Bosch BNO055 erledigt die Sensorfusion vollständig on-chip und liefert direkt eine fertige absolute Orientierung; das könnte den Komplementärfilter in `PoseEstimator` überflüssig machen, ist aber eine Architekturentscheidung für den Moment, in dem der konkrete Chip feststeht. Ob die in Abschnitt 5 beschriebene Komplementärfilter-Fusion mit einem MPU-6050-artigen Gyro auf der Zielhardware ebenso funktioniert, muss dort geprüft werden; in ArduinoHAL existiert noch keine dieser Bibliotheken.

Erreichbarkeit einer definierten Winkelgeschwindigkeit. Die Schnittstelle `setLeftSpeed/setRightSpeed(float radPerSec)` setzt, wie das geschwindigkeitsgeregelter Webots-Motormodell, eine gesteuerte (open-loop) Geschwindigkeitsvorgabe voraus. Ob sich damit auf den realen Getriebemotoren tatsächlich ein definiertes rad/s einstellt, ist offen; erste Erfahrungen zeigen ein unstetiges Ansprechverhalten der Motoren.

Zum tatsächlichen Fahrverhalten (Rückmeldung 16.09.2026): Der Coasterbot fährt Skid-Steer ohne separate Lenkung, Kurven entstehen ausschließlich über die Drehzahldifferenz zwischen linker und rechter Seite. Je Seite sind zwei TT-Getriebemotoren (Front und Heck) gekoppelt und werden wie ein einziger Kanal angesteuert, passend zu `setLeftSpeed/``setRightSpeed`. Solche Getriebemotoren werden typischerweise über eine H-Brücke (z. B. L298N, TB6612FNG, DRV8833) per PWM angesteuert; der PWM-Tastgrad ist dabei *kein* rad/s, sondern hängt zusätzlich von Akkuspannung, Last und Bodenreibung ab. Zwei Umsetzungen sind denkbar:

- **Open-loop:** einmalig eine Kennlinie Tastgrad zu rad/s ausmessen und fest hinterlegen. Einfach, driftet aber mit sinkender Akkuspannung und mit Fertigungsstreuung zwischen den Motoren, wodurch der Roboter bei nominell gleichem Befehl links/rechts leicht kurvt statt geradeauszufahren.
- **Closed-loop (empfohlen):** Falls nötig, müsste `ArduinoHAL::setLeftSpeed/setRightSpeed` intern eine Regelschleife führen, den einen Encoder je Seite lesen, die Ist-Geschwindigkeit schätzen und die Motoransteuerung per P- oder PI-Regler nachführen, sodass die `RobotHAL`-Schnittstelle für alle aufrufenden Komponenten unverändert bliebe. Die Regelung wäre vollständig in der Hardwaretreiberebene gekapselt.

Diese Regelschleife ist noch nicht umgesetzt, es gibt noch keine `ArduinoHAL`. Zu beachten bei der Verdrahtung: Hängen beide Motoren einer Seite an einem gemeinsamen H-Brücken-Kanal, addiert sich ihr Stromverbrauch, das Rating des Treiber-ICs muss dafür ausreichen.

Der Zusammenhang zur Lokalisierung: Skid-Steer-Räder rollen beim Drehen nicht sauber ab, sie schrammen seitlich über den Boden. Genau deshalb vertraut `PoseEstimator` dem Gyro beim Kurs bereits deutlich stärker als der Radodometrie (`HEADING_ODO_WEIGHT` = 0,05, siehe Abschnitt 5).

5 Lokalisierungskomponente

Die Lokalisierung (`PoseEstimator`, Dateien `pose_estimator.{h,cpp}`) bestimmt fortlaufend die Pose des Roboters in der Tisebene und erfüllt NAV-006 (Positionsbestimmung) und NAV-007 (Orientierungsbestimmung).

5.1 Prinzip

Der Coasterbot *misst* seine Position nicht, sondern **führt sie durch Koppelnavigation (Dead Reckoning) fort**. Es gibt keine externe Referenz: kein GPS, keine Bake, keine kamerabasierte Lokalisierung. Die Pose entsteht in vier Schritten:

1. **Bekannte Startpose.** `main.cpp` setzt den Schätzer über `pose.reset(x, y,  $\theta$ )` auf die im Szenario/in der Welt definierte Startposition. Alles Weitere ist relativ zu diesem Punkt.
2. **Radodometrie.** In jedem Regelzyklus wird über `RoboTHAL::getWheelAngle` je ein Encoderwert pro Seite gelesen (`WHEEL_LEFT/WHEEL_RIGHT`, die reale Elektronik hat nur einen Encoder je Seite) und mit dem Radradius in eine gefahrene Strecke umgerechnet.
3. **Kursfusion.** Die Kursänderung wird aus der Radodometrie *und* der integrierten Gyro-Giergeschwindigkeit über einen Komplementärfilter bestimmt, überwiegend aus dem Gyro.
4. **Integration.** Strecke und Kurs werden zur neuen Pose (x, y, θ) aufsummiert.

5.2 Verfahren

Verwendet wird eine **Skid-Steer-Odometrie** aus den zwei seitenweisen Radwinkeln, deren Kurs mit der integrierten Gyro-Giergeschwindigkeit über einen **Komplementärfilter** fusioniert wird. Die Alternativenbeschreibung nennt hierfür grundsätzlich EKF/UKF; für den Prototyp auf einer ebenen Tischfläche mit nur zwei Sensorquellen genügt der wesentlich einfachere Komplementärfilter, der ohne Kovarianzschätzung auskommt und deterministisch ist (wichtig für ST-SIM-004).

Aus dem Encoderwert je Seite folgt die gefahrene Strecke dieser Seite (in der Simulation mittelt `WebotsHAL` dafür intern die zwei PositionSensor von Front- und Heckrad derselben Seite, um dieselbe Information wie der eine reale Encoder je Seite nachzubilden; siehe Abschnitt 4.3):

$$\Delta s_L = (\varphi_L - \varphi_{L,\text{prev}}) r, \quad \Delta s_R = (\varphi_R - \varphi_{R,\text{prev}}) r$$

mit Radradius $r = 0.035$ m. Daraus folgen der Vorschub des Zentrums und die odometrische Kursänderung:

$$\Delta s = \frac{1}{2}(\Delta s_L + \Delta s_R), \quad \Delta \theta_{\text{odo}} = \frac{\Delta s_R - \Delta s_L}{b}, \quad b = 0.202 \text{ m (Spurweite)}.$$

Die fusionierte Kursänderung gewichtet Odometrie und Gyro:

$$\Delta \theta = w \Delta \theta_{\text{odo}} + (1 - w) \dot{\psi}_{\text{gyro}} \Delta t, \quad w = 0,05.$$

Da Skid-Steer beim Drehen stark schlupft, wird überwiegend dem Gyro vertraut ($w = 0,05$). `getGyroZ()` liefert dabei die Y-Komponente des Gyrovektors: Das Y-up-PROTO wird in der Z-up-Welt mit `rotation 1 0 0 1.5708` aufgestellt, wodurch die lokale +Y-Achse mit der Welt-Hochachse zusammenfällt.

Die Integration erfolgt über den mittleren Kurs des Schritts (Runge-Kutta 2. Ordnung):

$$x += \Delta s \cos\left(\theta + \frac{1}{2}\Delta\theta\right), \quad y += \Delta s \sin\left(\theta + \frac{1}{2}\Delta\theta\right), \quad \theta += \Delta\theta.$$

5.3 Schnittstelle

Methode	Bedeutung
<code>void update()</code>	einmal pro Regelzyklus nach <code>hal.step()</code>
<code>float getX() / getY()</code>	Position [m]
<code>float getTheta()</code>	Kurs [rad], $(-\pi, \pi]$
<code>float getOdometer()</code>	gesamte gefahrene Strecke [m]
<code>void reset(x,y,θ)</code>	Pose auf bekannten Startwert der Welt setzen

Koordinatenkonvention: x nach vorn (Startausrichtung), y nach links, θ positiv gegen den Uhrzeigersinn. Der absolute Gierwinkel der InertialUnit (`getYaw()`) wird im Schätzer bewusst *nicht* verwendet, da die 90°-Aufstellrotation die IMU nahe den Gimbal-Lock in der Nickachse bringt.

5.4 Grenzen der Lokalisierung

- **Driftakkumulation.** Reine Koppelnavigation summiert Messfehler auf; ohne absolute Korrektur oder Schleifenschluss (loop closure) wächst der Positionsfehler mit der gefahrenen Strecke. Für die kleine Tischfläche und die kurzen Fahrten der Simulationstests ist das ausreichend.
- **Manuelles Umsetzen.** Wird der Roboter körperlich angehoben und versetzt, ist die Schätzung anschließend falsch, bis `reset()` erneut eine bekannte Pose vorgibt, entsprechend der Einschränkung in NAV-006 („solange er nicht manuell umgesetzt wurde“).
- **Verifikation.** MODE_ODOMETRY_TEST auf `coasterbot-testfield.wbt`: Geradeausfahrt 0.699m gegenüber 0.70m Soll, kein seitlicher Versatz; Drehung auf der Stelle wird vom Gyro sauber verfolgt; im Stillstand bleibt die Pose stabil.

6 Umgebungsmodell

OccupancyGrid (Dateien `occupancy_grid.{h,cpp}`) entspricht der im Pflichtenheft genannten „bekannten Tischkarte“ und ist die gemeinsame Eingabe für A*, D* Lite und DWA. Es setzt SIM-004 um (Hindernisse in der Simulation berücksichtigen).

6.1 Aufbau

- Achsenparalleles Rechteck in **Weltkoordinaten** (X-Y-Ebene der Webots-Welt, Z = oben) mit fester Auflösung (Standard 0.05 m). Es liegt in derselben Ebene wie die Pose.

- In den Simulationsszenarien ist das Gitter auf die Tischfläche begrenzt, damit geplante Wege die Tischkante nicht überschreiten.
- Zellen sind *frei* oder *belegt*; Zellen außerhalb der Karte gelten als belegt.

6.2 Funktionen

Funktion	Wirkung
<code>addObstacleRect(..., infl)</code>	Rechteck-Hindernis, um <code>infl</code> Meter aufgebläht eintragen (Roboter-Halbbreite $\Rightarrow$ Punktroboter-Planung, erfüllt NAV-010)
<code>setOccupied(Cell, bool)</code>	einzelne Zelle HART setzen; Rückgabe <code>true</code> bei Änderung (für inkrementelle Neuplanung)
<code>reportOccupied(Cell, smoothingFactor)</code>	einzelne Zelle GEGLÄTTET melden (Abschnitt 6.3)
<code>buildDistanceField()</code>	Chamfer-Distanzfeld über zwei Rasterdurchläufe berechnen
<code>clearance(Vec2)</code>	Abstand eines Weltpunkts zum nächsten Hindernis [m]
<code>worldToCell</code> / <code>cellToWorld</code>	Umrechnung Welt $\leftrightarrow$ Zelle

Das Distanzfeld wird nach jeder Hindernisänderung als veraltet markiert und muss vor der nächsten `clearance()`-Abfrage neu gebaut werden. In `MODE_SIM_TEST` und `MODE_NAVIGATE_DSTAR` werden *zwei* Gitter geführt: ein großzügig aufgeblähtes Planer-Gitter (hält den Pfad von Ecken fern) und ein nur um den Roboterradius aufgeblähtes DWA-Gitter (Kollisionsbewertung). Beide werden bei Sensortreffern gemeinsam aktualisiert.

6.3 Geglättete Live-Hindernismeldung

`addObstacleRect()` trägt a priori bekannte Hindernisse hart und sofort ein (z. B. die bekannte Bierflasche im Simulationstest). Das ist gerechtfertigt, weil diese Eingabe nicht verrauscht ist. Ein per Ultraschall oder Kantensensor während der Fahrt *live* erkanntes Hindernis ist dagegen eine einzelne Sensormessung und kann durch Rauschen oder eine kurze Fehlmessung verursacht sein. Bis Version 5 dieses Dokuments hätte bereits ein einzelner solcher Treffer die Karte sofort verändert (`setOccupied(c, true)`) und damit eine D*-Lite-Umplanung ausgelöst.

`reportOccupied(Cell, smoothingFactor)` glättet das: statt die Zelle sofort zu belegen, wird eine Belegungs-Konfidenz $\text{conf} \in [0, 1]$ je Zelle exponentiell nachgeführt,

$$\text{conf} += \text{smoothingFactor} \cdot (1 - \text{conf}),$$

und die Zelle erst dann tatsächlich belegt (wie `setOccupied`), wenn `conf ≥ OCC_COMMIT_THRESHOLD` (Anhang B). Mit dem Standardwert `smoothingFactor = 0,5` genügen bereits drei aufeinanderfolgende Treffer an derselben Zelle (bei 32ms Regelzyklus also ≈ 100 ms), ein einzelner Ausreißer reicht dagegen nicht. `main.cpp` verwendet `reportOccupied()` inzwischen überall dort, wo bisher `setOccupied()` für Live-Sensortreffer aufgerufen wurde (`dstar_scenario::markRect`, `sim_scenario::markRect`).

Die schnelle Reflexbewegung bleibt davon unberührt: `DwaPlanner` bewertet die Kollisionsfreiheit einer Trajektorie weiterhin *jeden* Zyklus direkt aus dem rohen Ultraschallwert als virtuellem Hindernispunkt (Abschnitt 9.3), ungeglättet. Geglättet wird ausschließlich die *deliberative* Entscheidung, ob D* Lite die globale Karte dauerhaft ändert und umplant. Die sofortige Ausweichreaktion bei akuter Kollisionsgefahr bleibt so unverzögert.

Verifiziert: `MODE_SIM_TEST` und die Hinderniserkennung in `MODE_NAVIGATE_DSTAR` liefern nach der Umstellung dieselben Ergebnisse wie zuvor (ST-SIM-001..004 weiterhin BESTANDEN, byte-identisch über zwei Läufe; D*-Lite-Umplanung in `coasterbot-dstar.wbt` weiterhin bei $t \approx 3.8$ s, Ziel nach 24 s mit 1.5 cm Restfehler). Die zusätzliche Latenz durch die Glättung liegt unterhalb der Auflösung dieser Szenarien.

7 Globale Pfadplanung mit A*

`astarPlan(grid, start, goal)` (Dateien `astar.{h,cpp}`) berechnet einen globalen Fahrweg auf der bekannten Karte. Umgesetzte Anforderungen: NAV-008 (Zielpunkt erreichen), NAV-009 (autonome Bewegung), NAV-010 (Sicherheitsabstand).

7.1 Algorithmus

- 8-fach verbundenes Gitter, euklidische Heuristik, Prioritätswarteschlange.
- Kein „corner cutting“: eine Diagonale ist nur befahrbar, wenn auch die beiden orthogonalen Nachbarzellen frei sind.
- Liegen Start oder Ziel in einer aufgeblähten Hinderniszone, wird per Ringsuche die nächste freie Zelle verwendet.
- **Glättung („string pulling“)**: `smoothPath()` behält von jedem Anker den entferntesten noch per Sichtlinie erreichbaren Wegpunkt. Das vermeidet die 45°-Treppen des Rohpfades.
- **Zentrierung**: `centerPath()` verschiebt innere Wegpunkte senkrecht zur Fahrtrichtung in Richtung größeren Hindernisabstands (max. 0.20 m), damit der Pfad in engen Korridoren mittig verläuft. Ohne diesen Schritt bleibt der DWA-Planer an eckennahen Pfaden hängen.

Rückgabe: Wegpunkte in Weltkoordinaten von Start bis Ziel; leerer Vektor bedeutet „kein Pfad“.

7.2 Auswahlbegründung

Gemäß Alternativenbeschreibung standen Dijkstra und BFS als Alternativen zur Wahl. A* wurde wegen der zielgerichteten Heuristik gewählt: Bei gleicher Optimalität (zulässige Heuristik) expandiert A* deutlich weniger Knoten als Dijkstra. Für die kleine Tischkarte ($\sim 32 \times 40$ Zellen) ist die Erstplanung damit in Bruchteilen einer Millisekunde abgeschlossen.

8 Dynamische Neuplanung mit D* Lite

DStarLite (Dateien `dstar_lite.{h,cpp}`) repariert den Pfad inkrementell, wenn während der Fahrt ein Hindernis auftaucht oder verschwindet. Umgesetzte Anforderung: NAV-005 (Fahrtroute während der Bewegung anpassen).

8.1 Verfahren

Implementiert ist D* Lite nach Koenig & Likhachev (2002):

- Rückwärtssuche vom Ziel; je Zelle ein g -Wert (bekannte Kosten) und ein rhs -Wert („one-step lookahead“). Eine Zelle ist konsistent, wenn $g = rhs$.
- `std::set`-basierte Prioritätswarteschlange über Schlüssel (k_1, k_2) ; der Modifikator k_m akkumuliert die Heuristikverschiebung, während der Roboter fährt, sodass die Warteschlange ohne vollständige Neusortierung gültig bleibt.
- `plan(start, goal)` führt eine *vollständige* Erstexpansion der erreichbaren Karte durch (`expandAll`). Ein vollständiges g -Feld sorgt dafür, dass eine spätere lokale Änderung auch dann repariert wird, wenn der Ausweichweg zuvor nicht „auf dem Weg“ lag.
- `setStart(cell)` bei jedem Zellwechsel des Roboters; `cellChanged(cell)` bei jeder Belegungsänderung stellt die betroffenen Knoten neu in die Warteschlange; `computeShortestPath()` repariert dann nur den betroffenen Bereich.
- `extractPath()` folgt vom Start greedy den kleinsten g -Werten zum Ziel; `pathExists()` und `finiteGCount()` dienen der Diagnose.
- Ist der gesamte geplante Korridor durchtrennt (inkrementelle Reparatur liefert keinen Pfad), setzt `main.cpp` D* Lite aus der aktuellen Pose vollständig neu auf, was immer noch günstiger ist als eine Neuplanung in jedem Zyklus.

Alternative laut Alternativenbeschreibung war LPA*; D* Lite ist die auf bewegliche Startpositionen zugeschnittene Variante von LPA* und daher für einen fahrenden Roboter die passende Wahl.

9 Lokale Kollisionsvermeidung und Bewegungssteuerung mit DWA

DwaPlanner (Dateien `dwa_planner.{h,cpp}`) folgt dem globalen Pfad und wählt in jedem Regelzyklus ein sicheres Geschwindigkeitspaar (v, ω) . Er vereint die Pflichtenheft-Komponenten „lokale Kollisionsvermeidung“ und „Bewegungssteuerung“ und setzt NAV-001, NAV-002, SAF-003 und SAF-004 um.

9.1 Dynamic Window Approach

1. **Dynamisches Fenster:** Aus der aktuellen Geschwindigkeit und den Beschleunigungsgrenzen ($a_v = 0.5 \text{ m s}^{-2}$, $a_\omega = 6 \text{ rad s}^{-2}$) wird der in einem Zeitschritt erreichbare (v, ω) -Bereich bestimmt und in 6×25 Kandidaten abgetastet. Grenzen: $v \in [0, 0.18 \text{ m s}^{-1}]$, $|\omega| \leq 2.6 \text{ rad s}^{-1}$.
2. **Vorwärtssimulation:** Jeder Kandidat wird über $T_{\text{sim}} = 1.5 \text{ s}$ (Schritt 0.1 s) kinematisch simuliert.
3. **Verwerfen:** Trajektorien mit Kollision oder solche, aus denen der Roboter nicht mehr bremsen kann ($v > \sqrt{2 a_v \text{ clearance}}$), werden verworfen. Damit ist SAF-004 (keine schädigende Anfahrgeschwindigkeit) erfüllt.
4. **Bewertung:** Die verbleibenden Kandidaten werden gewichtet bewertet:

$$S = w_h S_{\text{Kurs}} + w_p S_{\text{Fortschritt}} + w_c S_{\text{Abstand}} + w_v S_{\text{Tempo}}$$

mit $w_h = 1,0$, $w_p = 2,0$, $w_c = 1,5$, $w_v = 0,3$. S_{Kurs} bewertet die Ausrichtung zum Carrot-Punkt, $S_{\text{Fortschritt}}$ die Annäherung an ihn, S_{Abstand} den Hindernisabstand entlang der Trajektorie, S_{Tempo} die Fahrgeschwindigkeit.

5. **Sackgasse:** Ist kein Kandidat zulässig, dreht der Roboter langsam auf der Stelle, bis wieder Optionen entstehen.

9.2 Pfadverfolgung (Pure Pursuit)

Der **Carrot-Punkt** ist der Punkt 0.30 m entlang des Polygonzugs vor der Projektion der Roboterposition auf den Pfad. Da er stets auf dem kollisionsfrei geglätteten Pfad liegt, bleibt die globale Hindernisumfahrung erhalten, während DWA die lokale, dynamikkonforme Ausführung übernimmt.

9.3 Hindernisse

DWA bezieht Hindernisse aus *zwei* Quellen:

- dem **Distanzfeld** der (bereits aufgeblähten) Belegungskarte: hier wird nur ein kleiner Restpuffer `GRID_MARGIN = 0.03 m` abgezogen;

- dem **aktuellen Ultraschallwert** als virtuellem Hindernispunkt vor dem Roboter: ein roher Punkt, von dem der volle Roboterradius 0.14 m abgezogen wird. Damit werden unkartierte bzw. dynamische Hindernisse abgedeckt (NAV-001).

Über `setUltrasonicRange(r)` lässt sich der Ultraschalleinfluss begrenzen: Trägt eine übergeordnete Ebene Hindernisse selbst in die Karte ein, wirkt DWA nur noch als Nahbereichs-Sicherung (MODE_SIM_TEST: $r = 0.45$ m).

9.4 Bewegungssteuerung

`driveVW(v,  $\omega$ )` rechnet das gewählte Geschwindigkeitspaar in Radgeschwindigkeiten um

$$\omega_L = \frac{v - \omega b/2}{r}, \quad \omega_R = \frac{v + \omega b/2}{r}$$

($r = 0.035$ m, $b = 0.202$ m) und gibt sie über `RoboTHAL::setLeftSpeed / setRightSpeed` aus. Die Geschwindigkeitsbegrenzung (MAX_V), das kontrollierte Anhalten am Ziel (GOAL_TOL = 0.05 m) und der sofortige Stopp bei fehlendem Pfad decken die im Pflichtenheft geforderten Eigenschaften der Bewegungssteuerung ab.

Alternative laut Alternativenbeschreibung waren Potentialfelder; diese wurden wegen der Gefahr lokaler Minima nicht gewählt. DWA umgeht das Problem durch die explizite Trajektorienbewertung und die Kopplung an den globalen Pfad.

10 Sicherheitskomponente

`SafetyMonitor` (Dateien `safety_monitor.{h,cpp}`) ist die im Lasten- und Pflichtenheft geforderte **Sicherheitskomponente / Sicherheitslogik** mit übergeordneter Priorität. Umgesetzte Anforderungen: NAV-003, NAV-004, SAF-001, SAF-002 (Struktur), SAF-003, SAF-005.

10.1 Funktion

In jedem Regelzyklus (*vor* der Navigation) wertet der Monitor die vier Kantensensoren aus. Erkennt ein Sensor die Tischkante, hält er sofort an, übernimmt die Aktorik und führt eine Ausweichbewegung aus:

Situation	Reaktion
Kante vorne	Rückwärtsfahrt ($\text{EVADE_RADS} = 3 \text{ rad s}^{-1}$, $\text{EVADE_TIME} = 0.7 \text{ s}$)
Kante hinten	Vorwärtsfahrt
Kante vorne <i>und</i> hinten	Stillstand (Zustand PINNED)

Zustandsautomat: `CLEAR` $\rightarrow$ `EVADE` $\rightarrow$ `PAUSE` (0.3 s) $\rightarrow$ `CLEAR`; bei beidseitiger Kante `PINNED`. Solange `overriding()` wahr ist, darf die Navigation *keine* eigenen Fahrbefehle geben. Das stellt `main.cpp` sicher, indem der Regelzyklus nach einem haltenden `safety.update()` vorzeitig endet.

10.2 Schnittstelle

Methode	Bedeutung
<code>bool update()</code>	<code>true</code> , solange die Komponente die Aktorik hält
<code>bool overriding()</code>	Zustand $\neq$ <code>CLEAR</code>
<code>bool edgeEventThisTick()</code>	steigende Flanke einer Kantenerkennung
<code>bool frontEdge()</code> / <code>rearEdge()</code>	betroffene Seite
<code>const char* stateName()</code>	aktueller Zustand (Systemüberwachung, <code>MON-003</code>)
<code>int eventCount()</code>	Zähler aller Kantenergebnisse

Not-Aus (SAF-005). Der Software-seitige Anker ist die sofortige Aktorübernahme mit `drive(0)`. Der physische Not-Aus-Taster (`HW-BTN-003`) wird über dieselbe Abstraktions-Schnittstelle als Ereignis eingespeist und ist in der Simulation nicht modelliert.

Sensorausfall (SAF-002). Die Zustandsstruktur für den Wechsel in einen sicheren Zustand ist vorhanden; die gezielte Fehlerinjektion (`ST-SIM-005`) ist noch nicht implementiert (siehe Abschnitt 16).

11 Reaktive Verhaltenslogik

`RobotLogic` (Dateien `robot_logic.{h,cpp}`) ist ein einfacher reaktiver Zustandsautomat zur Hindernis- und Kantenvermeidung. Er diente der Inbetriebnahme der HAL und ist die Grundlage der Betriebsart `MODE_AUTONOMOUS`. Für die zielgerichtete Navigation wird er durch den Navigationsstapel (`A*` / `D*` Lite / `DWA`) ersetzt.

Zustand	Verhalten
<code>FORWARD</code>	geradeaus mit <code>CRUISE_SPEED</code> = 6 rad s ⁻¹ ; bei Kante $\rightarrow$ <code>REVERSE</code> , bei Hindernis (< 0.22 m) $\rightarrow$ <code>AVOID_OBSTACLE</code>
<code>REVERSE</code>	0.6 s zurücksetzen, dann <code>TURN</code>
<code>TURN</code>	auf der Stelle drehen (zufällige Richtung), bis frei
<code>AVOID_OBSTACLE</code>	drehen, bis der Ultraschall wirklich frei ist

Zusätzlich stellt RobotLogic die expliziten Bewegungsprimitive `forward`, `backward`, `turnLeft`, `turnRight` und `stop` bereit, die in den Test-Betriebsarten direkt aufgerufen werden.

12 Ablaufsteuerung und Betriebsarten

Die Anwendungsebene (`main.cpp`) instanziiert `WebotsHAL`, `RobotLogic` und `PoseEstimator` und wählt über den Kompilierschalter `static const Mode MODE` eine Betriebsart. So ist jederzeit nachvollziehbar, welche Bewegung warum ausgeführt wird (NFA-TST-*: gezielt erzeugbare, reproduzierbare Abläufe).

Tabelle 11: Betriebsarten des Controllers.

MODE	Zweck	Welt
MODE_WHEEL_TEST	je Seite 2 s die Räder drehen, Encoder ausgeben	beliebig
MODE_MANUAL	feste Befehlsfolge über die Bewegungsprimitive	beliebig
MODE_AUTONOMOUS	reaktiver Zustandsautomat RobotLogic	coasterbot.wbt
MODE_ODOMETRY_TEST	definierte Fahrt, Pose ausgeben (Odometrieprüfung)	coasterbot-testfield.wbt
MODE_NAVIGATE	A*-Pfad planen und mit DWA abfahren	coasterbot-astar.wbt
MODE_NAVIGATE_DSTAR	D* Lite + per Ultraschall entdecktes Hindernis → Umplanung	coasterbot-dstar.wbt
MODE_SIM_TEST	kombinierter Simulationstest, Bewertung ST-SIM-001..004	coasterbot-simtest.wbt

12.1 Ablauf MODE_SIM_TEST

1. Pose auf die Startpose der Welt setzen; Planer- und DWA-Gitter aus dem Szenario `sim_scenario` aufbauen (Tischrand als Absturzkante, bekannte Bierflasche auf der Route, Deko in den Ecken).
2. D* Lite Erstplanung; Pfad glätten und zentrieren.
3. **Phase 1 (Navigation, ST-SIM-001/002):** Sicherheitskomponente → Kartenpflege → D* Lite → DWA, bis das Ziel innerhalb der Toleranz erreicht ist. Kleinster Hindernisabstand und Anzahl der Umplanungen werden protokolliert.

4. **Phase 2 (Tischkante, ST-SIM-003):** Roboter richtet sich auf $+Y$ aus und fährt langsam Richtung Tischkante; nur die Sicherheitskomponente darf noch eingreifen. Es wird gezählt, wie oft sie die Kante erkennt und ob der Roboter auf dem Tisch bleibt.
5. Bewertung ausgeben (BESTANDEN / FEHLGESCHLAGEN je Testfall). ST-SIM-004 wird durch zweimaligen Lauf und Vergleich der Ausgabe geprüft (byte-identisch, da alle Komponenten deterministisch sind).

13 Simulationsmodell

Das Simulationsmodell erfüllt NFA-SIM-001..005 und SIM-001..007: Die Robotersoftware läuft unverändert; nur die hardwarenahe Schicht wird durch Webots-Modelle ersetzt (NFA-SIM-004).

13.1 Robotermodell (PROTO)

`protos/coasterbot/Coasterbot.proto` ist aus der CAD-Baugruppe abgeleitet.

- **Koordinaten:** PROTO ist Y-up (lokal $+X$ rechts, $+Y$ oben, $-Z$ vorne). Die Welten sind Z-up (ENU); Instanzen stehen mit `rotation 1 0 0 1.5708` aufrecht.
- **Antrieb:** vier `RotationalMotor`, je Seite als `coupled motors` gekoppelt (`wheel_left::front` mit `wheel_left::rear`, analog rechts).
- **Sensoren:** vier `PositionSensor` an den Radgelenken (Encoder); **ultrasonic** (`DistanceSensor`, sonar, `lookupTable` in Metern, **analoges** Distanzsignal); vier `edge_*` (`DistanceSensor`, infrarot, nach unten, **digitales** Signal: `lookupTable 00 / 0,0980 / 0,101 / 41`, eine reine 0/1-Stufe mit 2 mm numerischer Übergangszone statt einer analogen Rampe, siehe Abschnitt 4.2); **inertial_unit**, **gyro**, **accelerometer**.
- **Kollisionskörper:** bewusst zwei gestapelte Boxen (schmäler unterer Teil zwischen den Rädern) gegen Zittern.

Die `lookupTables` der Distanzsensoren wurden so kalibriert, dass die Software mit demselben Signaltyp wie auf der Zielhardware arbeitet: Ultraschall liefert weiterhin ein analoges Distanzsignal in Metern (Vergleich mit `OBSTACLE_THRESHOLD_M`); die Kantensensoren liefern ein digitales 0/1-Signal, das `WebotsHAL` nur noch auf seine Polarität hin abfragt (`EDGE_ACTIVE_HIGH`, siehe Abschnitt 4.2); ein analoger Schwellenwert steht dafür nicht mehr im Code.

13.2 Welten

Tabelle 12: Simulationswelten mit Bezug zur Software.

Welt	Inhalt / zugehörige Betriebsart
coasterbot.wbt	Tisch mit Flaschen; <code>MODE_AUTONOMOUS</code>
coasterbot-testfield.wbt	offene 4×4 -Fläche; <code>MODE_ODOMETRY_TEST</code>
coasterbot-astar.wbt	zwei versetzte Wände (Slalom), Zielmarker; <code>MODE_NAVIGATE</code>
coasterbot-dstar.wbt	Wand mit zwei Lücken, unkartierte Box; <code>MODE_NAVIGATE_DSTAR</code>
coasterbot-simtest.wbt	Table $1,6 \times 2,0$, FlowerPot + BeerBottles; <code>MODE_SIM_TEST</code>

Die Szenario-Konstanten in `main.cpp` (Namensräume `nav_scenario`, `dstar_scenario`, `sim_scenario`) müssen mit der jeweiligen Welt übereinstimmen (Start, Ziel, Hindernispositionen, Tischgrenzen).

13.3 Reproduzierbarkeit

SIM-001 / ST-SIM-004 werden erfüllt, weil alle Komponenten deterministisch sind: fester Zeitschritt, keine Zufallszahlen im Navigationsstapel (nur `RobotLogic` nutzt `std::rand` und ist an der Navigation nicht beteiligt), sortierte Prioritätswarteschlangen mit eindeutigen Schlüsseln. Zwei kopflose Läufe von `MODE_SIM_TEST` liefern byte-identische Protokolle.

14 Systemüberwachung

Die Systemüberwachung (MON-001..005) ist in der Simulation als strukturiertes Konsolenprotokoll umgesetzt. Jede Betriebsart gibt aus:

- **Betriebszustand** (MON-003): Zustandswechsel von `RobotLogic` (`[AUTONOMOUS]` ...Zustand: ...) und `SafetyMonitor` (`stateName()`), Pose alle 2s bis 3s.
- **Ereignisse**: Kantenerkennung, Erkennung eines unkartierten Hindernisses, Umpfanung, Zielerreichung.
- **Fehlererkennung** (MON-001): „KEIN PFAD“, Zeitüberschreitung der Navigation, Wächterabbruch (`guard hit`) in `DStarLite::computeShortestPath`.
- **Aufgabenergebnis** (MON-004/005): in `MODE_SIM_TEST` pro Testfall `BESTANDEN` bzw. `FEHLGESCHLAGEN` mit Kennzahlen (Restfehler zum Ziel, minimaler Hindernisabstand, Kantenergebnisse, „Tisch verlassen“).

Nicht umgesetzt sind eine persistente Protokolldatei mit Zeitstempeln und die im Pflichtenheft skizzierte zentrale Ereignisverwaltung; das Konsolenprotokoll deckt den Prototyp-Bedarf ab.

15 Abbildung auf Anforderungen und Testfälle

Tabelle 13: Rückverfolgbarkeit Anforderung → Softwarekomponente → Test.

ID	Umsetzung	Nachweis
NAV-001	DwaPlanner (Ultraschall als virtuelles Hindernis), OccupancyGrid	ST-SIM-002
NAV-002	DwaPlanner, DStarLite	ST-SIM-002
NAV-003	SafetyMonitor (Kantensensoren)	ST-SIM-003
NAV-004	SafetyMonitor; Gitter auf Tischfläche begrenzt	ST-SIM-003
NAV-005	DStarLite + Kartenpflege in main.cpp	ST-SIM-002
NAV-006	PoseEstimator (Odometrie)	ST-SIM-001
NAV-007	PoseEstimator (Gyro-Fusion)	ST-SIM-001
NAV-008	astarPlan / DStarLite + DwaPlanner	ST-SIM-001
NAV-009	gesamter Navigationsstapel	ST-SIM-001/002
NAV-010	OccupancyGrid-Inflation, DWA-clearance, SafetyMonitor	ST-SIM-002 (min. Abstand)
NAV-011	<i>nicht umgesetzt</i> (keine Getränkeerkennung)	keine
SAF-001	SafetyMonitor (sofortiger drive(0))	ST-SIM-003
SAF-002	SafetyMonitor-Struktur; Fehlerinjektion offen	(ST-SIM-005)
SAF-003	DwaPlanner (Kollisions-/Bremsprüfung), SafetyMonitor	ST-SIM-002/003
SAF-004	DwaPlanner: MAX_V + Bremsbedingung	ST-SIM-002
SAF-005	SafetyMonitor-Übernahme; Taster über HAL-Ereignis	keine
MON-001	Konsolenprotokoll, „KEIN PFAD“, Wächter, Zeitüberschreitung	ST-SIM-*
MON-002	stdout-Protokoll (keine Datei)	ST-SIM-*
MON-003	getStateName() / stateName(), Pose-Ausgabe	ST-SIM-*

ID	Umsetzung	Nachweis
MON-004	DwaPlanner::reachedGoal(), Verdikt p1	ST-SIM-001
MON-005	FEHLGESCHLAGEN-Ausgabe, Zeitüberschreitung	ST-SIM-*
NFA-ARC-001/002	je Komponente eine Klasse/Datei, klare Trennung	Inspektion
NFA-ARC-003	HAL + entkoppelte Planer; neue Komponenten ohne Kernänderung	Inspektion
NFA-ARC-004	RobotHAL, setPath, update, clearance	Inspektion
NFA-ARC-005	RobotHAL / WebotsHAL	Inspektion
NFA-MAI-001/002	nur webots_hal.cpp ist hardwareabhängig	Inspektion
NFA-MAI-004	Komponenten nehmen nur RobotHAL / OccupancyGrid	Inspektion
NFA-SIM-001..005	Webots-Modell, WebotsHAL, identische Schnittstellen	ST-SIM-*
SIM-001	deterministische Komponenten	ST-SIM-004
SIM-004	OccupancyGrid + Welt-Hindernisse	ST-SIM-002
SIM-005	offen (Fehlerinjektion)	ST-SIM-005/006

15.1 Status der Simulationstestfälle

Testfall	Gegenstand	Status
ST-SIM-001	Navigation zu einem Zielpunkt	umgesetzt, MODE_SIM_TEST Phase 1
ST-SIM-002	Dynamische Hindernisvermeidung	umgesetzt (bekannte + unkar- tierte Hindernisse)
ST-SIM-003	Verhindern des Tischverlassens	umgesetzt, MODE_SIM_TEST Phase 2
ST-SIM-004	Wiederholbarkeit	umgesetzt (zwei Läufe, byte- identisch)
ST-SIM-005	Sensorausfall	offen (Fehlerinjektion fehlt)
ST-SIM-006	Aktorfehler	offen
ST-SIM-007	Untersetzerhandling	außerhalb des Umfangs (keine Manipulation)
ST-SIM-008	Validierung Simulation vs. Prototyp	offen (Prototyp erforderlich)

16 Grenzen und offene Punkte

- **Missionsablauf:** Der Zustandsautomat `IDLE` $\rightarrow \dots \rightarrow$ `DONE` ist nicht implementiert; die Software fährt zielgerichtet zu einem vorgegebenen Punkt bzw. reaktiv.
- **Untersetzerhandling / Manipulation:** `CST-*` sowie `IK + RRT*` sind nicht Teil der Simulationssoftware.
- **Lokalisierung:** statt EKF/UKF ein Komplementärfilter; ausreichend für die ebene Tischfläche, aber ohne Unsicherheitsschätzung.
- **Ultraschall:** ein einzelner schmaler Vorwärtssensor findet kleine, leicht außeraxiale Tischobjekte nicht zuverlässig. Daher sind im Simulationstest Blumentopf und Flaschen in der Karte hinterlegt („vermessener Tisch“); die reine Sensor-Entdeckung wird mit einem großen Hindernis in `MODE_NAVIGATE_DSTAR` gezeigt.
- **Zwei Gitter + centerPath:** Übergangslösung, damit DWA eckennahen Kürzestpfaden folgen kann. Eine weiche (gradientenbasierte) Kostenkarte würde beide Gitter zu einem zusammenführen.
- **Fehlerinjektion:** `ST-SIM-005/006` (Sensor-/Aktorfehler) und die persistente Ereignisprotokollierung stehen aus.
- **Validierung gegen den Prototyp:** `ST-SIM-008` erfordert den realen Coasterbot.
- **Hardware-Realisierbarkeit von RobotHAL:** Eignung von `getGyroZ()` sowie die offene Frage, ob `setLeftSpeed/setRightSpeed` auf realen Motoren eine Regelschleife benötigen; siehe die Rückmeldung des Hardware-Teams in Abschnitt 4.3 (Radgranularität und Kantensensoren sind aus derselben Rückmeldung bereits umgesetzt: `enum WheelId` auf `WHEEL_LEFT/WHEEL_RIGHT` reduziert, Kantensensoren als `bool`-Schnittstelle).

16.1 Ausblick: initiale Tischerkundung

Als nächster funktionaler Ausbauschritt ist eine **initiale Tischerkundung** vorgesehen (Rückmeldung 14.09.2026): Der Coasterbot fährt einmalig (vor dem eigentlichen Bedienbetrieb) der Reihe nach alle vier Ecken der Tischfläche an, legt an jeder Ecke einen Getränkeuntersetzer ab und kehrt anschließend zu seinem Ausgangspunkt zurück. Ziel ist es, damit für den nachfolgenden Betrieb *bekannte* Untersetzerpositionen zu schaffen, sodass Untersetzer gezielt (an einer bekannten Position statt per Objekterkennung) abgelegt und wieder aufgenommen werden können.

Diese Funktion ist noch **nicht implementiert**. Der nachfolgende Stufenplan (Stand 14.09.2026) beschreibt, was für eine Umsetzung erforderlich wäre; er dient als Grundlage für die spätere Umsetzung, nicht als bereits getroffene Architekturentscheidung.

Stufe 0: Vorab zu klärende Entscheidungen.

- **Realitätsgrad der Ablage:** entweder ein tatsächlich in der Szene erscheinendes Coaster-Objekt (Supervisor-basiert, empfohlen, konsistent zu

Blumentopf/Flaschen, die bereits als echte Szenenobjekte behandelt werden) oder zunächst nur ein protokolliertes Ablage-*Ereignis* ohne sichtbare Änderung der Szene.

- **Reihenfolge der Ecken:** feste Abfolge (empfohlen, deterministisch, passt zum Determinismus-Anspruch aus ST-SIM-004) oder Nächster-Nachbar-Heuristik.
- **Welt:** eine neue, aufgeräumte `coasterbot-explore.wbt` (Tisch ohne die Bierflasche/den Blumentopf aus `coasterbot-simtest.wbt`) statt einer weiteren Überladung der bestehenden Simulationstest-Welt.

Stufe 1: Coaster-PROTO. Ein neues, einfaches PROTO (`protos/coaster/Coaster.proto`): eine dünne Scheibe mit definierten Abmessungen (Lastenheft-Anforderung an Untersetzer), ohne Kollisionskörper, da das Objekt per Supervisor platziert und nicht angefahren wird.

Stufe 2: Schnittstelle CoasterHandling. Eine *eigene*, von RobotHAL getrennte Abstraktion: „Objekt in der Szene erzeugen/verschieben“ hat keine Entsprechung auf realer Hardware, gehört also nicht in die HAL, die portierbar bleiben soll. Entspricht der im Pflichtenheft ohnehin separat geführten Komponente „Untersetzerhandling“.

```
class CoasterHandling {
public:
    virtual ~CoasterHandling() {}
    // Legt einen (virtuellen) Untersetzer an der Position ab.
    // Rueckgabe: false, wenn kein Untersetzer mehr verfuegbar ist.
    virtual bool placeCoaster(Vec2 at) = 0;
    // Fuer eine spaetere Aufnahme-Phase; in dieser Stufe ungenutzt.
    virtual bool pickUpCoaster(Vec2 at) = 0;
};
```

`WebotsCoasterHandling` implementiert dies über die Webots-Supervisor-API (`importMFNodeFromString` auf den Szenenwurzelknoten, das zurückgegebene `Node*` je Ecke für eine spätere Aufnahme gemerkt). Dafür muss `WebotsHAL` intern von `webots::Robot` auf `webots::Supervisor` wechseln (Supervisor *ist* ein Robot, rein additiv; die RobotHAL-Schnittstelle selbst ändert sich nicht) und die jeweilige `Coasterbot`-Instanz in der Welt `supervisor TRUE` setzen (ein Feld, das das PROTO bereits besitzt, aber bislang ungenutzt lässt, siehe Abschnitt 13.1).

Stufe 3: Eckpunkt-Geometrie. Neuer Namensraum `explore_scenario` in `main.cpp`: vier Zielposen, von jeder Tischkante um den Montageversatz der Kantensensoren im PROTO ($x = \pm 0,09$, $z = \mp 0,09$) plus einen Sicherheitspuffer nach innen versetzt, jeweils mit Blick zur Tischmitte ausgerichtet, sodass nie gleichzeitig eine Kante vorne *und* hinten in Reichweite ist, was `SafetyMonitor` sonst in den Zustand `PINNED` (Abschnitt 10) treiben könnte. Der genaue Versatz muss empirisch ermittelt und nicht nur rechnerisch festgelegt werden.

Stufe 4: Ablaufsteuerung. Eine neue Funktion `runExploreTable()`, nach demselben Muster wie die Phasenschleife in `runSimTest()` (Abschnitt 12.1): eine Liste der vier Eckpunkte plus Ausgangspunkt, je Ziel D*-Lite-Planung + `DwaPlanner`, bei Zielerreichung `placeCoaster()` (nur an den ersten vier Zielen, nicht auf dem Rückweg), danach das nächste Ziel. `SafetyMonitor` bleibt wie gehabt vorrangig; ein Auslösen nahe einer Ecke sollte in dieser frühen Stufe zunächst protokolliert statt hart als Fehlschlag gewertet werden, um Stufe 3 iterativ nachjustieren zu können.

Stufe 5: Anbindung. Neuer Wert `MODE_EXPLORE_TABLE` im Mode-Enum, die Welt `coasterbot-explore.wbt`, sowie `webots_coaster_handling.cpp` im Makefile.

Stufe 6: Test und Rückverfolgbarkeit. Ein neuer Testfall (z.B. ST-SIM-009, im Testkonzept bislang nicht definiert) und die ersten Umsetzungsreferenzen für CST-001/CST-003 in Tabelle 13.

Stufe 7: Verifikation. Zweifacher kopfloser Lauf (Reproduzierbarkeit), Kontrolle, dass `SafetyMonitor` an keiner Ecke ungewollt auslöst, dass alle vier Untersetzer erscheinen und der Roboter innerhalb der Toleranz zum Ausgangspunkt zurückkehrt.

17 Anhang A: Schnittstelle RobotHAL

```
enum WheelId { WHEEL_LEFT = 0, WHEEL_RIGHT = 1 }; // ein Encoder je Seite

class RobotHAL {
public:
    virtual ~RobotHAL() {}

    virtual bool step() = 0; // ein Regelzyklus

    virtual void setLeftSpeed(float radPerSec) = 0; // linke Seite (gekoppelt)
    virtual void setRightSpeed(float radPerSec) = 0; // rechte Seite (gekoppelt)

    virtual float getUltrasonicDistance() = 0; // [m]
    virtual bool getEdgeFrontLeft() = 0; // true = Kante, kalibriert in der HAL
    virtual bool getEdgeFrontRight() = 0;
    virtual bool getEdgeRearLeft() = 0;
    virtual bool getEdgeRearRight() = 0;

    virtual float getYaw() = 0; // [rad]
    virtual float getGyroZ() = 0; // [rad/s]
    virtual float getWheelAngle(WheelId wheel) = 0; // [rad] EINER SEITE, + =
        vorwaerts
    virtual float getTime() = 0; // [s]
};
```

18 Anhang B: Wesentliche Parameter

Komponente	Parameter	Wert
Geometrie	Radradius r	0.035 m
	Spurweite b	0.202 m
	Radstand	0.113 m
PoseEstimator	HEADING_ODO_WEIGHT	0,05
OccupancyGrid	Auflösung	0.05 m
	OCC_COMMIT_THRESHOLD (Abschnitt 6.3)	0,8
	DEFAULT_SMOOTHING	0,5
astar	centerPath maxShift	0.20 m
DwaPlanner	MAX_V / MAX_W	$0.18 \text{ m s}^{-1} / 2.6 \text{ rad s}^{-1}$
	ACC_V / ACC_W	$0.5 \text{ m s}^{-2} / 6 \text{ rad s}^{-2}$
	SIM_TIME / SIM_DT	1.5 s / 0.1 s
	Gewichte w_h, w_p, w_c, w_v	1,0 / 2,0 / 1,5 / 0,3
	CARROT_DIST / GOAL_TOL	0.30 m / 0.05 m
	ROBOT_RADIUS	0.14 m
WebotsHAL	EDGE_ACTIVE_HIGH (Kantensensor-Polarität)	wahr
	EVADe_RADS / EVADe_TIME	$3 \text{ rad s}^{-1} / 0.7 \text{ s}$
SafetyMonitor	PAUSE_TIME	0.3 s
	OBSTACLE_THRESHOLD_M	0.22 m
RobotLogic	CRUISE_SPEED / TURN_SPEED	$6 \text{ rad s}^{-1} / 4 \text{ rad s}^{-1}$